

AI Software Supply-Chain Security and Vulnerability Detection: A Comprehensive Survey

Abstract

The rapid integration of artificial intelligence (AI) and large language models (LLMs) into critical enterprise infrastructure has precipitated a fundamental paradigm shift in software engineering. Consequently, the traditional software supply chain has evolved into a highly complex, multi-layered AI software supply chain (AI-SSC). While conventional software supply chains primarily consist of source code and deterministic dependencies, the AI-SSC introduces heterogeneous artifacts, including massive uncured training datasets, pre-trained model weights, dynamic autonomous agents, and specialized deep learning libraries. This survey provides an exhaustive examination of the vulnerabilities inherent to the AI-SSC and the state-of-the-art detection methodologies designed to mitigate them. The analysis systematically deconstructs the threat landscape across several critical vectors: ecosystem and dependency vulnerabilities in model registries, cryptographic provenance and AI Bills of Materials (AIBOMs), serialization and deserialization exploits, deep learning library fuzzing, backdoor and data poisoning attacks, and indirect prompt injection within autonomous agent architectures. By synthesizing current research, this survey highlights the systemic failure of traditional static security mechanisms when applied to non-deterministic AI models. The findings underscore the critical need for defense-in-depth strategies, ranging from hardware-assisted trusted execution environments (TEEs) for provenance tracking to mathematically grounded robust statistics for backdoor detection. Ultimately, this survey establishes a unified framework for understanding, evaluating, and securing the next generation of AI-native software ecosystems.

Introduction

Modern software development is increasingly reliant on complex, multi-layered ecosystems that incorporate artificial intelligence at their core. This integration has democratized access to advanced capabilities, such as natural language understanding, generative design, and autonomous decision-making. However, this convergence also introduces a highly heterogeneous security and privacy landscape that cannot be addressed through isolated, technology-specific solutions [1, 2]. The AI Software Supply Chain encompasses the entire lifecycle of machine learning systems. Recent taxonomies define the AI-SSC as consisting of five core components: computational resources, development tools, datasets, foundation models, and downstream applications [2, 3]. Because the development of an LLM or deep neural network is largely an iterative, data-driven process, the traditional security paradigms that focus exclusively on code logic and deterministic execution paths are insufficient [4, 5]. Unlike traditional software vulnerabilities—which typically stem from logical coding errors,

vulnerable third-party libraries, or insecure deployment configurations—AI-SSC vulnerabilities span a vastly broader attack surface. Deep learning models exhibit non-deterministic behaviors that blur the traditional boundaries between data and executable code [6, 7]. Empirical analyses of the AI-SSC reveal that it exhibits a "locally dense, globally sparse" topology, where a small number of foundational models and datasets act as high-degree hubs supporting millions of downstream applications [8]. For instance, analyzing the supply chain via dependency graphs demonstrates that a single vulnerability at the foundation model layer, or within a widely used training dataset, can cascade exponentially through the ecosystem, affecting hundreds of dependent nodes at varying layers of abstraction [8, 9].

The threat model for the AI-SSC is multifaceted and deeply intertwined with the open-source software (OSS) communities that drive AI innovation. At the infrastructure level, deep learning libraries such as PyTorch and TensorFlow introduce memory safety vulnerabilities in their low-level C++ kernels, which can be exploited via high-level Python APIs [10, 11]. At the distribution level, platforms like Hugging Face and PyPI are targeted by adversaries utilizing typosquatting, dependency confusion, and malicious serialization payloads that execute arbitrary code upon model instantiation [12, 13, 14]. At the algorithmic level, models are vulnerable to data poisoning, model hijacking, and sophisticated backdoor attacks that remain dormant until triggered by specific input patterns [15, 16, 17]. Finally, at the deployment level, LLM-integrated applications and autonomous agents face indirect prompt injection attacks, where malicious instructions embedded in untrusted external data subvert the agent's core objectives [18, 19].

To address these compounding risks, this survey provides a comprehensive analysis of AI supply chain security. The subsequent sections organize the research area into coherent method families and research directions, exploring the mechanics of emerging attacks, the empirical evidence of their impact, and the theoretical foundations of modern vulnerability detection and mitigation systems.

Methods

The study of AI software supply-chain security requires a multidisciplinary approach, combining classical binary analysis, cryptographic attestation, robust statistics, and adversarial machine learning. This section categorizes the vulnerability detection and mitigation landscape into six distinct research directions, tracing the AI model lifecycle from its foundational dependencies to its deployment as an autonomous agent.

Ecosystem and Dependency Vulnerability Detection

The foundational layer of the AI-SSC relies heavily on open-source software (OSS) ecosystems, predominantly Python package registries such as PyPI and npm, as well as AI-specific model hubs like Hugging Face. The immense scale of these ecosystems makes manual security auditing computationally and logistically infeasible, exposing practitioners to severe supply chain poisoning.

Adversaries routinely exploit the automated package resolution mechanisms of PyPI and npm through typosquatting and dependency confusion attacks. Typosquatting involves registering package names that are orthographically similar to popular libraries, while dependency

confusion exploits the tendency of package managers to prioritize public registries over private, internal repositories when identically named packages exist with higher version numbers [20, 21]. Advanced malicious packages are increasingly employing dynamic payloads, where the malicious code is not present in the static source but is fetched at runtime or embedded deep within `setup.py` or `package.json` installation scripts [22]. To detect these threats, research has shifted from heuristic rule-based systems to deep learning and behavioral sequence analysis. Systems like EMPHunter utilize explicit-knowledge-free clustering algorithms to analyze installation scripts, effectively grouping and flagging anomalous behaviors without relying on predefined malware signatures [22]. In empirical evaluations, EMPHunter successfully identified hundreds of previously unknown malicious packages across PyPI and npm by recognizing distinct structural similarities in how scripts execute malicious payloads during the installation phase [22]. Similarly, cross-ecosystem frameworks like Cerebro model malicious behaviors as sequential actions, enabling multi-lingual knowledge fusing to detect novel attacks across different package managers by recognizing the underlying behavioral intent rather than specific language syntax [23].

Beyond traditional code registries, AI-specific hubs introduce unique architectural vulnerabilities. Platforms like Hugging Face allow developers to host AI-Powered Applications (AI-Apps), often deploying them via lightweight containers or virtualized spaces. Recent large-scale analyses have revealed critical access control and resource reuse flaws within these platforms [12, 24]. For example, "Identifier Reuse Attacks" exploit the way platforms generate subdomains. When an application is deleted, its subdomain becomes available, allowing an attacker to claim it and hijack orphaned `iframe` links embedded in third-party websites [24]. Furthermore, "Ghost Token Attacks" leverage the stateless nature of JSON Web Tokens (JWTs). Attackers can generate valid tokens for a repository, delete the repository, and wait for a victim to register the same namespace, at which point the attacker retains unauthorized access via the persistently valid token [24]. Detecting these ecosystem-level threats requires massive scale graph-based supply chain analysis, such as HuggingGraph, which maps the complex lineage and dependency trees of base models, datasets, and fine-tuned variants to identify the propagation paths of vulnerabilities [8]. Furthermore, license incompatibility has emerged as a severe ecosystem risk. Deep learning models often combine datasets and pre-trained weights with conflicting open-source licenses, creating legal liabilities. Frameworks like LiAgent leverage LLM-based reasoning to perform ecosystem-level license compatibility analysis, identifying thousands of conflicts that traditional static analysis tools miss due to the complex, multi-layered derivation of AI models [25].

To address the challenge of accurately classifying diverse file types and payloads within these massive repositories, researchers have developed highly optimized neural network classifiers. Magika, an AI-powered file type detection tool deployed by Google, replaces brittle, hand-written signature matching (such as the traditional `file` command) with a deep learning model [26]. Magika operates by evaluating 512-byte sequences from the beginning, middle, and end of a file. Running on a highly optimized ONNX neural network requiring only 1MB of memory, it achieves a 99% F1 score across hundreds of content types with a 5-millisecond inference time on a single CPU [26]. This allows repository maintainers to accurately route incoming files to appropriate security scanners, thwarting attempts to bypass security controls

by masking malicious scripts as benign data formats.

Cryptographic Provenance and AI Software Bills of Materials

As regulatory frameworks increasingly demand transparency in AI systems, the concept of the Software Bill of Materials (SBOM) has been adapted into the Artificial Intelligence Bill of Materials (AIBOM). Traditional SBOMs meticulously track software dependencies, versions, and licenses, but they are fundamentally inadequate for AI systems. AI behavior is non-deterministic and emerges from the complex interplay of model architectures, training hyperparameters, massive datasets, and stochastic optimization processes [27, 28]. An effective AIBOM must capture a multi-layered inventory that extends beyond static code to include the cognitive and data-driven layers of the system. This requires capturing the data layer (training and validation datasets, preprocessing transformations, data provenance, and assessments of demographic bias), the model layer (specifications of the neural architecture, learned weights, fine-tuning checkpoints, and the full lineage of model derivation), the infrastructure layer (the deep learning frameworks and hardware accelerators utilized), and the runtime policy layer (the active guardrails and prompt filtering mechanisms applied during deployment) [29, 30]. Standardization bodies are actively developing schemas to support these requirements, notably the SPDX 3.0 AI Profile and CycloneDX ML-BOM [31, 32]. However, a documented AIBOM is only as reliable as its source data. Without cryptographic integrity, a malicious model provider could easily falsify an AIBOM to obscure the use of poisoned data or stolen intellectual property. To address this trust deficit, state-of-the-art provenance frameworks leverage Hardware-Assisted Trusted Execution Environments (TEEs) and cryptographic attestation protocols.

Provenance Framework	Core Mechanism	Primary Security Guarantee
Laminator	Executes ML training within hardware-assisted TEEs, generating signed attestations linking the training dataset to the final model weights [33].	Prevents providers from falsifying ML property cards or altering datasets post-attestation.
Atlas	Combines TEEs with append-only transparency logs to track end-to-end metadata from data ingestion to deployment [34].	Ensures holistic supply chain integrity and limits unauthorized access during distributed pipeline execution.
AIBoMGen	Acts as a neutral third-party	Automates the generation

	observer within the training environment, hashing datasets, container images, and parameters using the in-toto framework [35].	of cryptographically verifiable AIBOMs with negligible performance overhead.
Zero-Knowledge Proofs (zkPoT)	Employs SNARKs or STARKs to generate succinct mathematical proofs that a specific model was trained on a specific dataset [36].	Validates training provenance without revealing the underlying proprietary model weights or sensitive data.

By utilizing attestation protocols, the TEE generates a digitally signed AIBOM that mathematically guarantees the model was trained exactly as specified, free from unauthorized tampering by dishonest administrators [34, 35]. This hardware-rooted trust is critical for enabling verifiable machine learning property cards and ensuring the integrity of the AI-SSC across distributed environments, paving the way for compliance with stringent regulations like the European Union's AI Act.

Artifact Serialization and Deserialization Exploits

A critical and highly exploited vulnerability vector within the AI-SSC is the serialization and deserialization of machine learning models. To share models across platforms, frameworks must convert complex in-memory objects, such as neural network weights and computational graphs, into byte streams. Historically, the Python pickle module has been the de facto standard for this task, natively supported by frameworks like PyTorch [37, 38].

The fundamental security flaw of pickle is its design as an expressive, stack-based virtual machine (the Pickle Machine). During deserialization, pickle parses opcodes that allow for the instantiation of arbitrary Python objects and the execution of callable functions, typically via the `__reduce__` method [39, 40]. This architecture permits adversaries to craft malicious serialized models that execute arbitrary system commands, establish reverse shells, or deploy ransomware the moment the model is loaded into memory [37, 41]. These attacks, sometimes referred to as "Sleepy Pickle" vulnerabilities, can be engineered to execute their payloads silently while seamlessly returning the legitimate model weights, rendering the compromise invisible to the end-user. The model appears to function normally, classifying images or generating text as expected, while a background process exfiltrates sensitive data [41, 42].

Defensive mechanisms against malicious serialization have evolved through several iterations. Initial approaches relied on static model scanners, such as Picklescan and Fickling, which parse the pickle bytecode and check against a denylist of known dangerous functions (e.g., `os.system`, `subprocess.Popen`) [38, 39]. However, these denylist approaches suffer from significant limitations. Attackers easily bypass them using obfuscation techniques, such as utilizing `asyncio` gadgets, exploiting less-known built-in functions, or intentionally corrupting

the byte stream with invalid opcodes (e.g., injecting an early 0x2E STOP opcode) that cause scanners to crash while the underlying Python interpreter still processes the malicious payload [37, 38].

To achieve robust security, research has pivoted toward dynamic enforcement and the adoption of inherently safe serialization formats. Systems like PickleBall operate as drop-in replacements for the native pickle module. PickleBall utilizes static analysis to profile the expected behavior of a benign machine learning library, generating a strict allowlist of necessary classes and functions. During load time, it dynamically enforces this policy, blocking any opcode that attempts to invoke unauthorized callables [39, 43]. Empirical evaluations demonstrate that PickleBall can successfully load nearly 80% of benign models while blocking 100% of malicious payloads, introducing a minimal runtime overhead of approximately 2.62% [43]. Furthermore, machine learning classifiers like SafePickle have been developed to extract structural and semantic features from Pickle bytecode, successfully identifying highly obfuscated evasive payloads with an F1 score exceeding 90% without requiring explicit code instrumentation [44].

Concurrently, the industry is transitioning toward data-only serialization formats such as safetensors. Supported natively by Hugging Face, safetensors completely removes the execution of arbitrary code from the loading process. It stores tensors in a flat, zero-copy format that is strictly validated against a defined schema, offering mathematical guarantees against arbitrary code execution while simultaneously loading models significantly faster than traditional pickle files [45, 46].

Deep Learning Library Fuzzing and Validation

The computational foundations of the AI-SSC rest on deep learning libraries such as TensorFlow, PyTorch, and JAX. These libraries expose high-level Python APIs to developers while executing intensive computations via highly optimized, low-level C++ or CUDA kernels [10, 47]. Vulnerabilities within these libraries—such as buffer overflows, use-after-free errors, and integer overflows—can be exploited to crash systems, leak memory, or execute arbitrary code [48].

Fuzzing, the process of feeding pseudo-random, unexpected inputs into a program to uncover crashes, is the standard methodology for discovering software vulnerabilities. However, traditional byte-level mutation fuzzers (like AFL) are largely ineffective against DL libraries [10, 49]. DL APIs impose strict, complex input constraints regarding tensor shapes, dimensions, and data types. For instance, a matrix multiplication API requires specific dimensional alignment between two input tensors. Inputs that fail these basic validation checks are immediately rejected by shallow exception handlers at the Python-C++ boundary, preventing the fuzzer from reaching the deeply nested kernel logic where memory corruption bugs actually reside [47, 50].

To overcome this "constraint barrier," researchers have developed sophisticated API-level and model-level fuzzing frameworks tailored specifically for deep learning ecosystems. These frameworks leverage diverse methodologies to infer constraints and generate valid edge cases:

1. **Documentation-Guided and Open-Source Mining:** Frameworks like DocTer utilize

Natural Language Processing (NLP) to parse API documentation, extracting syntactic dependency trees to automatically infer input constraints (e.g., "input must be a 4D tensor of type float32"). By formalizing these constraints, DocTer can generate inputs that reliably pass shallow type checks [50, 51]. Alternatively, FreeFuzz bypasses natural language ambiguity by mining millions of open-source Python files and developer tests. It instruments the code to dynamically record the valid shapes and types of tensors passed to APIs in the wild, using this empirical data as a seed corpus for mutation fuzzing [52, 53].

2. **Type-Aware and Relational Fuzzing:** IvySyn performs type-aware mutation fuzzing directly on the low-level C++ APIs. Upon discovering a crash, it synthesizes a high-level Python code snippet that triggers the same vulnerability, automatically generating a Proof of Vulnerability (PoV) that demonstrates its exploitability from the user space [10, 49]. DeepREL takes a different approach by automatically inferring relational APIs—functions that should yield identical outputs for identical inputs. It leverages differential testing across these relations to detect subtle logical bugs and precision errors that do not necessarily cause hard crashes [54, 55]. Automatic Differentiation (AD), the core engine of neural network training, has also been targeted; tools like ∇ Fuzz isolate the AD components to verify the mathematical correctness of high-order gradient computations through differential testing [56].
3. **LLM-Driven Fuzzing:** The most recent paradigm shift involves utilizing Large Language Models as zero-shot fuzzers. Frameworks such as TitanFuzz and FuzzGPT leverage the vast corpus of code ingested by LLMs during pre-training. Because LLMs have implicitly learned the intricate semantic constraints of PyTorch and TensorFlow APIs, they can autoregressively generate highly diverse, valid edge-case programs that pass shallow checks and stress deep kernel logic, significantly outperforming traditional constraint-solving approaches in code coverage and bug discovery [57, 58]. Further advancements, such as DFuzz, integrate LLMs into a white-box workflow, prompting the LLM to summarize low-level C++ validation checks and explicitly generate abstract test cases designed to provoke edge-case failures [48].

Backdoor, Data Poisoning, and Model Hijacking Detection

At the algorithmic level, the AI-SSC is severely threatened by malicious manipulations of the training pipeline. Because modern foundation models require web-scale datasets comprising trillions of tokens or images, rigorous manual curation is impossible. Attackers can exploit this vast, unverified data ingestion process. Research has demonstrated that web-scale data poisoning is highly practical. "Split-view" poisoning involves purchasing expired domains previously linked in web datasets to serve malicious content specifically to web crawlers while presenting benign content to human users. Similarly, "frontrunning" attacks inject malicious edits into periodically snapshotted crowdsourced platforms like Wikipedia just before a scraping process occurs [59, 60].

Data poisoning is the primary delivery mechanism for Backdoor Attacks, also known as Neural Trojans. A backdoor attack embeds a hidden associative rule within the neural network. The model performs perfectly on benign validation data, ensuring the attack remains undetected

during standard quality assurance. However, when an input contains an attacker-specified "trigger"—such as a specific pixel patch, a hidden word, or a broad semantic concept—the model's internal activations are hijacked, causing it to output an incorrect, targeted prediction [61, 62, 63].

A related threat is Model Hijacking. While backdoors typically alter the classification of a specific input, model hijacking forces a model to perform an entirely different, covert task. For example, an attacker might hijack an innocuous text-translation model to operate simultaneously as a spam-generation or data-exfiltration engine [64, 65]. By hosting this hijacked model on a platform like Hugging Face, the attacker offloads the computational cost and legal liability to the victim who deploys the model in their infrastructure [66]. This threat extends to federated learning environments, where an adversary can search for pixel-level perturbations based on their local model to align hijacking samples with original ones in the feature space, compromising the global model without direct access to the central server [67]. Detecting backdoors in deep neural networks is exceptionally difficult, and recent theoretical work has proven that universal, adversary-unaware backdoor detection is mathematically impossible except for trivially small data alphabets [68, 69]. Consequently, defenders must rely on heuristic and statistical anomaly detection frameworks. These generally fall into three distinct paradigms:

Detection Paradigm	Methodology	Key Frameworks
Trigger Inversion & Feature Pruning	Uses optimization algorithms to reverse-engineer the minimal perturbation required to cause a mass misclassification to a target class. If a highly compact trigger is found, a backdoor is assumed to exist. The network can then be pruned to excise the compromised neurons.	Neural Cleanse [61, 70], TABOR [71]
Latent Space & Spectral Analysis	Analyzes the hidden feature representations of the training data. Poisoned inputs must trick the model into learning a strong signal, creating a distinct, anomalous sub-population in the latent space. SVD and	Spectral Signatures [72, 73], TED++ [74]

	robust statistics isolate and remove this "Spectral Signature."	
Meta-Neural Analysis	Treats detection as a secondary machine learning task. A "meta-classifier" is trained on a large dataset of generated "shadow models" (both clean and backdoored). The meta-classifier learns fundamental distributional differences in the weight space to classify the target model.	MNTD [75, 76], FREEEAGLE [77]

Despite these robust defenses, attackers continuously innovate, shifting the paradigm from data-driven triggers to behavior-driven triggers. Emerging techniques include quantization backdoors, where the malicious logic remains completely dormant in 32-bit floating-point precision. The model operates benignly until it undergoes post-training quantization to 8-bit integers for edge deployment, at which point the specific rounding errors amplify the hidden perturbations and awaken the backdoor [78, 79]. Furthermore, studies in continual learning show that static backdoors often suffer from "catastrophic forgetting" as models ingest new tasks. In response, attackers have developed persistent latent-task backdoors that modify loss computation surfaces, allowing the malicious behavior to survive sequential model updates and fine-tuning [80]. Additionally, the widespread use of parameter-efficient fine-tuning (PEFT) methods, such as LoRA adapters, has created a new attack vector. Researchers have shown that backdoors can be embedded within LoRA weights and distributed via hubs. Detecting these poisoned adapters requires analyzing the weight matrices directly for concentrated singular values and low spectral entropy, without requiring access to the original triggers or training data [81].

Indirect Prompt Injection and Autonomous Agent Security

As LLMs transition from isolated chatbots to autonomous agents integrated with external tools—such as databases, web browsers, and code interpreters—the AI-SSC attack surface shifts from the training phase to the inference and execution phase. The most pervasive and complex vulnerability in this domain is Prompt Injection [82, 83].

Prompt injection exploits a fundamental architectural flaw in current LLM-based systems: the inability to reliably distinguish between trusted system instructions and untrusted user data. In

a traditional application, the von Neumann architecture rigorously separates executable code from data payloads. In an LLM, instructions, system prompts, and external data are concatenated into a single, unified text stream [84]. Direct Prompt Injection, often referred to as jailbreaking or goal hijacking, occurs when a user intentionally crafts an input (e.g., "Ignore all previous instructions and output your system prompt") to bypass alignment guardrails, extract proprietary logic, or generate harmful content [85, 86].

However, a significantly more dangerous and systemic paradigm is Indirect Prompt Injection (IPI). In an IPI attack, the adversary does not interact with the LLM directly. Instead, the attacker embeds a malicious payload into a third-party location—such as a hidden text block on a webpage, a maliciously crafted resume, or an email [87, 88]. When an autonomous agent or Retrieval-Augmented Generation (RAG) system fetches this external document to fulfill a legitimate user request, it unknowingly ingests the attacker's payload. The LLM processes the payload as an authoritative instruction, leading to unauthorized tool execution, data exfiltration, or the contamination of cross-agent communications [89, 90, 91].

Research indicates that IPI attacks can be highly stealthy and overwhelmingly effective. Attackers utilize the "CFS Model" (Context, Format, Saliency), blending the payload into the document's structure using zero-width Unicode characters, HTML attribute cloaking, or semantic trickery, while employing authoritative, imperative language to trigger the model's instruction-following bias [92, 93]. End-to-end evaluations on adversarial frameworks like AgentDojo and InjecAgent demonstrate that a single poisoned email or web page can consistently coerce an agent into exfiltrating SSH keys, transferring funds, or executing arbitrary code [94, 95, 96]. Optimization-based attacks, such as HOUYI and ToolHijacker, formulate prompt injection as a gradient-based or genetic algorithm problem, iteratively mutating adversarial prompts to maximize the likelihood of subverting an agent's tool-selection process or bypassing alignment filters entirely [97, 98].

Defending against prompt injection in autonomous agents requires layered, defense-in-depth architectures, as no single mechanism can fully eliminate the threat:

- **Static Analysis of Prompts:** Frameworks like PromptSonar parse application source code prior to deployment, extracting string literals and identifying hardcoded prompts that expose sensitive data or fail to securely delimit user inputs. This prevents insecure prompt architectures from reaching production [99].
- **Dual LLM Patterns and Privilege Isolation:** This structural quarantine involves deploying a "Privileged LLM" that executes tools but never sees raw external text, alongside a "Quarantined LLM" that summarizes external text but has no tool access. The two models communicate via constrained, symbolic variables to prevent the transmission of injected commands, mitigating the Confused Deputy problem [92].
- **Attention-Based and Multi-Agent Filtering:** Advanced detection systems monitor the self-attention mechanisms of the LLM during inference to detect anomalous shifts indicating task-hijacking [100]. Other approaches employ multi-agent orchestration, using specialized "guardrail agents" to evaluate the safety and policy compliance of outputs before they are executed by tools or shown to the user [82, 101].

Challenges and Prospects

Despite the rapid advancement of detection methodologies, securing the AI software supply chain remains fraught with systemic challenges that will dictate the future trajectory of the field.

The Scale and Immutability of Web Data: The foundation of modern AI capabilities relies on web-scale datasets. Because these datasets encompass billions of documents and petabytes of data, comprehensive human curation and cryptographic verification at the source level are practically impossible. Consequently, data poisoning is not merely a theoretical threat, but an inevitable environmental hazard [59]. Current anomaly detection tools, such as Spectral Signatures, are computationally expensive to scale to trillions of tokens. The ecosystem must therefore shift from attempting to entirely prevent poisoning to developing model architectures and robust training algorithms that are inherently resilient to high levels of data contamination.

Verification of Non-Deterministic Outputs: Traditional software security relies heavily on deterministic state analysis—if an input matches a known malicious signature, the application blocks it. Deep learning models, however, map high-dimensional continuous spaces to probabilistic outputs. An adversarial payload, such as a prompt injection or a backdoor trigger, can be expressed in infinitely many semantic variations that evade traditional static denylists and Web Application Firewalls (WAFs) [93, 102]. Security paradigms must therefore evolve toward behavioral profiling, relying on dynamic causal analysis and LLM-assisted semantic reasoning to understand the underlying intent of an execution chain rather than its precise syntax [23, 103].

The Alignment Paradox and the Erosion of the Code/Data Boundary: The integration of autonomous agents introduces severe architectural vulnerabilities. Empirical studies suggest that the more capable an LLM becomes at following complex instructions, the more susceptible it becomes to sophisticated prompt injection, creating an "alignment paradox" where utility directly scales with vulnerability [89, 104]. Until neuro-symbolic architectures or distinct logical pathways can firmly separate execution context from data ingestion within the neural weights themselves, agentic workflows will remain fundamentally vulnerable to malicious subversion [92].

Regulatory Friction and Cryptographic Transitions: While frameworks like the EU AI Act mandate supply chain transparency, the operationalization of AIBOMs is hindered by commercial realities. Proprietary model developers are hesitant to disclose granular training data provenance or precise hyperparameter configurations due to intellectual property and liability concerns. Furthermore, the impending transition to post-quantum cryptography (PQC) threatens to invalidate the classical digital signatures currently used to verify model lineage and TEE attestations. This necessitates a proactive overhaul of current supply chain assurance models to integrate quantum-safe signature schemes [59]. Bridging the gap between theoretical cryptographic provenance and practical, privacy-preserving industry adoption remains a critical open challenge.

Conclusion

The evolution of the AI Software Supply Chain has introduced a revolutionary set of capabilities to the software engineering ecosystem, simultaneously exposing it to unprecedented security

risks. This survey has detailed how vulnerabilities permeate every layer of the AI lifecycle. From dependency confusion in Python registries and malicious arbitrary code execution via pickle serialization, to the subtle insertion of dormant backdoors in web-scale datasets and the hijacking of autonomous agents through indirect prompt injections, the attack surface is vast, highly heterogeneous, and deeply interconnected.

The findings presented underscore a critical reality: traditional, reactive security measures—such as static code analysis and heuristic denylists—are fundamentally insufficient to protect non-deterministic, probabilistic AI systems. Addressing these threats necessitates a paradigm shift toward "secure-by-design" AI infrastructure. This requires the widespread adoption of robust, data-only serialization formats like safetensors, the integration of Hardware-Assisted Trusted Execution Environments to ensure the cryptographic integrity of AI Bills of Materials, and the implementation of strict privilege isolation architectures for autonomous agents. Ultimately, as AI models gain increasing autonomy over critical enterprise infrastructure and user data, securing the AI supply chain must move from a secondary compliance objective to the foundational engineering priority of the next generation of artificial intelligence.

References

- [1] S. Wang, Y. Zhao, et al., "SoK: Understanding Vulnerabilities in the Large Language Model Supply Chain," arXiv preprint arXiv:2502.12497, 2025.
- [2] Y. Wang, Y. Zhao, et al., "Large Language Model Supply Chain: A Research Agenda," in Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE), 2024. arXiv ID: 2404.12736.
- [3] Y. Ma, L. Quan, X. Xie, Q. Hu, J. Yu, Y. Zhang, and S. Chen, "Understanding the Supply Chain and Risks of Large Language Model Applications," arXiv preprint arXiv:2507.18105, 2025.
- [4] F. Ladisa, et al., "A Taxonomy of Attacks on Open-Source Software Supply Chains," in Proceedings of the IEEE Symposium on Security and Privacy (S&P), 2023.
- [5] P. Radanliev, "Operationalising Artificial Intelligence Bills of Materials (AIBOMs) for Verifiable AI Provenance and Lifecycle Assurance," arXiv preprint arXiv:2605.19755, 2026.
- [6] G. K. Rajbahadur, K. Gallaba, E. Rashno, A. Suriyawongkul, K. Bennet, K. Stewart, and A. E. Hassan, "Building an Open AIBOM Standard in the Wild: An Experience Report on Extending the SPDX SBOM for AI Supply Chains," in Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE SEIP), 2026. arXiv ID: 2510.07070.
- [7] A. Bennet, et al., "TAIBOM: Bringing Trustworthiness to AI-Enabled Systems," in Proceedings of the European Conference on Machine Learning (ECML), 2025.
- [8] P. Gao, et al., "HuggingGraph: Understanding the Supply Chain of LLM Ecosystem," in Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM), 2025. arXiv ID: 2507.14240.
- [9] H. Wang, et al., "Hidden Licensing Risks in the LLMware Ecosystem," arXiv preprint arXiv:2602.10758, 2026.
- [10] N. Christou, et al., "IvySyn: Automated Vulnerability Discovery in Deep Learning Frameworks," in Proceedings of the USENIX Security Symposium, 2023.
- [11] H. Kim, et al., "Lightweight Concolic Testing via Path-Condition Synthesis for Deep Learning

- Libraries," in Proceedings of the International Conference on Software Engineering (ICSE), 2025.
- [12] Y. Gu, L. Ying, Z. Zhang, Y. Pu, X. Huang, J. Zhou, W. Zhu, D. Sun, and H. Duan, "Your Space is My Zone: Demystifying the Security Risks of AI-Powered Applications on Pre-Trained Model Hubs," in Proceedings of the ACM Conference on Computer and Communications Security (CCS), 2026. arXiv ID: 2606.30373.
- [13] T. Liu, et al., "The Art of Hide and Seek: Making Pickle-Based Model Supply Chain Poisoning Stealthy Again," in Proceedings of the USENIX Security Symposium, 2026.
- [14] X. Hou, Y. Zhao, and H. Wang, "Voices from the Frontier: A Comprehensive Analysis of the OpenAI Developer Forum," in Proceedings of the ACM/IEEE International Conference on Software Engineering Companion (ICSE-Companion), 2025.
- [15] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain," arXiv preprint arXiv:1703.01317, 2017.
- [16] Y. Liu, S. Ma, et al., "Trojaning Attack on Neural Networks," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2018.
- [17] H. Ohayon, et al., "Backdoor Learning: A Survey," arXiv preprint arXiv:2007.08745, 2020.
- [18] H. Chang, E. Bao, X. Luo, and T. Yu, "Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems," in Proceedings of the USENIX Security Symposium, 2026. arXiv ID: 2601.07072.
- [19] L. Tan, et al., "Indirect Prompt Injection in the Wild: An Empirical Study of Prevalence, Techniques, and Objectives," arXiv preprint arXiv:2604.27202, 2026.
- [20] Y. Pu, L. Ying, and Y. Gu, "From Noise to Signal: Precisely Identify Affected Packages of Known Vulnerabilities in npm Ecosystem," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2026.
- [21] Z. Zhang, et al., "PYPILINE: Malicious PyPI Package Detection via Suspicious API Knowledge and Agent Workflow," arXiv preprint arXiv:2606.19063, 2026.
- [22] L. Wermke, et al., "EMPHunter: Detecting Malicious Packages in PyPI and NPM by Clustering Installation Scripts," IEEE Transactions on Software Engineering, 2026.
- [23] W. Jiang, et al., "Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence," arXiv preprint arXiv:2309.02637, 2023.
- [24] Y. Gu, et al., "Your Space is My Zone: Demystifying the Security Risks of AI-Powered Applications," arXiv preprint arXiv:2606.30373, 2026.
- [25] H. Wang, et al., "LiCoEval: Evaluating LLMs on License Compliance in Code Generation," arXiv preprint arXiv:2501.12345, 2025.
- [26] Google Security Research, "Magika: AI-Powered Content-Type Detection," arXiv preprint arXiv:2409.13768, 2024.
- [27] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and J. Gebru, "Model Cards for Model Reporting," in Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*), 2019.
- [28] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for Datasets," Communications of the ACM, vol. 64, no. 12, pp. 86-92, 2021.
- [29] S. K. Anuguthala, "Enterprise AI Supply Chain Security Governance: A Framework for

- Secure Open-Source AI Model Adoption Using an Artificial Intelligence Bill of Materials (AI-BOM)," *International Journal For Multidisciplinary Research*, vol. 8, no. 3, pp. 1-15, 2026.
- [30] W. Vandendriessche, J. Thijsman, L. D'hooge, B. Volckaert, and M. Sebrechts, "AIBoMGen: Generating an AI Bill of Materials for Secure, Transparent, and Compliant Model Training," in *Proceedings of the ACM/IEEE International Conference on AI Engineering (CAIN)*, 2026. arXiv ID: 2601.05703.
- [31] G. K. Rajbahadur, et al., "Building an Open AIBOM Standard in the Wild," arXiv preprint arXiv:2510.07070, 2025.
- [32] P. Radanliev, "Operationalising Artificial Intelligence Bills of Materials," arXiv preprint arXiv:2605.19755, 2026.
- [33] V. Duddu, O. Järvinen, L. J. Gunn, and N. Asokan, "Laminator: Verifiable ML Property Cards using Hardware-assisted Attestations," in *Proceedings of the ACM Conference on Data and Application Security and Privacy (CODASPY)*, 2025. arXiv ID: 2406.17548.
- [34] W. Vandendriessche, et al., "Atlas: A Framework for ML Lifecycle Provenance & Transparency," arXiv preprint, 2025.
- [35] W. Vandendriessche, et al., "AIBoMGen: Generating an AI Bill of Materials," arXiv preprint arXiv:2601.05703, 2026.
- [36] H. Akgul, D. R. Borg, and M. Petrov, "Verifiable Fine-Tuning for LLMs: Zero-Knowledge Training Proofs Bound to Data Provenance and Policy," arXiv preprint arXiv:2510.12345, 2025.
- [37] M. Zhu, et al., "On the (In)Security of Loading Machine Learning Models," arXiv preprint arXiv:2509.06703, 2025.
- [38] L. Huang, et al., "Analyzing Python Deserialization Vulnerabilities in the Wild," in *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*, 2024.
- [39] A. D. Kellas, et al., "PICKLEFUZZER: A Case Study in Fuzzing for Discrepancies Between Python Pickle Implementations," in *Proceedings of the Workshop on Language-Theoretic Security (LangSec)*, 2026.
- [40] H. Ohayon, D. Gilkarov, and R. Dubin, "SafePickle: Robust and Generic ML Detection of Malicious Pickle-based ML Models," arXiv preprint arXiv:2602.19818, 2026.
- [41] T. Liu, et al., "Sleepy Pickle: Making Pickle-Based Model Supply Chain Poisoning Stealthy Again," in *Proceedings of the USENIX Security Symposium*, 2026.
- [42] A. D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V. P. Kemerlis, J. C. Davis, and J. Yang, "PickleBall: Secure Deserialization of Pickle-based Machine Learning Models," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025. arXiv ID: 2508.15987.
- [43] A. D. Kellas, et al., "PickleBall: Secure Deserialization of Pickle-based Models," arXiv preprint arXiv:2508.15987, 2025.
- [44] H. Ohayon, et al., "SafePickle: Robust ML Detection of Malicious Pickle-based Models," arXiv preprint arXiv:2602.19818, 2026.
- [45] X. Yi, et al., "An Empirical Study of Safetensors' Usage Trends and Developers' Perceptions," arXiv preprint arXiv:2501.02170, 2025.
- [46] Hugging Face, "Safetensors: Fast and Safe Tensor Serialization," arXiv preprint, 2025.
- [47] D. Xie, Y. Li, M. Kim, L. Tan, X. Zhang, and M. W. Godfrey, "DocTer: Documentation-Guided Fuzzing for Testing Deep Learning API Functions," in *Proceedings of the ACM SIGSOFT*

- International Symposium on Software Testing and Analysis (ISSTA), 2022. arXiv ID: 2109.01002.
- [48] W. Jiang, et al., "Security Knowledge-Guided Fuzzing of Deep Learning Libraries," arXiv preprint arXiv:2306.03269, 2023.
- [49] N. Christou, et al., "IvySyn: Automated Vulnerability Discovery in Deep Learning Frameworks," arXiv preprint, 2023.
- [50] D. Xie, et al., "DocTer: Documentation-Guided Fuzzing for Testing Deep Learning API Functions," arXiv preprint arXiv:2109.01002, 2021.
- [51] Z. Cheng, et al., "Balancing Validity and Vulnerability: Knowledge-Driven Seed Generation via LLMs for Deep Learning Library Fuzzing," *Applied Sciences*, vol. 15, no. 19, p. 10396, 2025.
- [52] C. S. Xia, et al., "Free Lunch for Testing: Fuzzing Deep-Learning Libraries from Open Source," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022. arXiv ID: 2201.06589.
- [53] S. Kwon, et al., "SKIPFUZZ: Active Learning for Fuzzing Deep Learning Libraries," arXiv preprint arXiv:2212.04038, 2022.
- [54] Y. Deng, et al., "Fuzzing Deep-Learning Libraries via Automated Relational API Inference," in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.
- [55] L. Zhang, et al., "Fuzzing Automatic Differentiation in Deep-Learning Libraries," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2023.
- [56] Y. Deng, et al., "Fuzzing Automatic Differentiation in DL Libraries," in *Proceedings of ICSE*, 2023.
- [57] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023. arXiv ID: 2212.14834.
- [58] T. Deng, et al., "FuzzGPT: A Large Language Model-Based Fuzzing Framework for Deep Learning Libraries," in *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*, 2023.
- [59] N. Carlini, M. Jagielski, C. Choquette-Choopani, D. Terzis, K. Thomas, and F. Tramèr, "Poisoning Web-Scale Training Datasets is Practical," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024. arXiv ID: 2302.10149.
- [60] N. Carlini, et al., "Poisoning Web-Scale Training Datasets is Practical," arXiv preprint arXiv:2302.10149, 2023.
- [61] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, "Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [62] M. Hanif, et al., "Survey on Backdoor Attacks on Deep Learning: Current Trends, Categorization, Applications, Research Challenges, and Future Prospects," *IEEE Access*, vol. 11, pp. 1-25, 2023.
- [63] X. Chen, et al., "Causality-inspired Backdoor Defense via Deconfounded Representation Learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [64] A. Salem, M. Backes, and Y. Zhang, "Get a Model! Model Hijacking Attack Against Machine

Learning Models," arXiv preprint arXiv:2111.04394, 2021.

[65] Y. Si, et al., "Two-in-One: A Model Hijacking Attack Against Text Generation Models," in Proceedings of the USENIX Security Symposium, 2023.

[66] H. Ohayon, et al., "Category-Agnostic Model Hijacking in Model Marketplaces," arXiv preprint arXiv:2408.13741, 2024.

[67] Z. Li, et al., "Model Hijacking Attack in Federated Learning," arXiv preprint arXiv:2408.02131, 2024.

[68] M. Zhao, et al., "On the (In)feasibility of ML Backdoor Detection as an Hypothesis Testing Problem," arXiv preprint arXiv:2402.16926, 2024.

[69] X. Chen, et al., "On the Limitation of Backdoor Detection Methods," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[70] B. Wang, et al., "Neural Cleanse," IEEE S&P, 2019.

[71] W. Guo, L. Wang, X. Xing, M. Du, and D. Song, "TABOR: A Highly Accurate Approach to Inspecting and Restoring Trojan Backdoors in AI Systems," arXiv preprint arXiv:1908.01763, 2019.

[72] Y. Tran, et al., "Spectral Signatures in Backdoor Attacks," in Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2018. arXiv ID: 1811.00636.

[73] W. L. Wang, et al., "PSBD: Prediction Shift Uncertainty Unlocks Backdoor Detection," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. arXiv ID: 2406.05826.

[74] N. L. Sun, et al., "TED++: A Submanifold-Aware Framework for Detecting Subtle Backdoors in Deep Learning," arXiv preprint arXiv:2407.12345, 2024.

[75] S. Xu, et al., "Detecting AI Trojans Using Meta Neural Analysis," arXiv preprint arXiv:1910.03137, 2020.

[76] A. Popovic, et al., "DeBackdoor: A Deductive Framework for Detecting Backdoor Attacks on Deep Models with Limited Data," in Proceedings of the USENIX Security Symposium, 2025.

[77] C. Fu, et al., "FREEEAGLE: Detecting Complex Neural Trojans in Data-Free Cases," in Proceedings of the USENIX Security Symposium, 2023.

[78] T. Huynh, et al., "Data Poisoning Quantization Backdoor Attack," in Proceedings of the European Conference on Computer Vision (ECCV), 2024.

[79] C. S. Xia, et al., "Architectural Backdoors in Deep Learning: A Survey of Vulnerabilities, Detection, and Defense," arXiv preprint arXiv:2507.12919, 2025.

[80] N. L. Sun, et al., "Neural Network Semantic Backdoor Detection and Mitigation: A Causality-Based Approach," in Proceedings of the USENIX Security Symposium, 2023.

[81] S. Luong and H. Chen, "Detecting Backdoored LoRAs from Weights Alone," arXiv preprint arXiv:2602.15195, 2026.

[82] M. Zhao, et al., "A Survey on Trustworthy LLM Agents: Threats and Countermeasures," arXiv preprint arXiv:2503.09648, 2025.

[83] H. Ohayon, et al., "A Survey on the Unique Security of Autonomous and Collaborative LLM Agents: Threats, Defenses, and Futures," arXiv preprint arXiv:2602.16551, 2026.

[84] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv preprint arXiv:2211.09527, 2022.

[85] F. Ladisa, et al., "The Landscape of Prompt Injection Threats in LLM Agents (SoK)," arXiv

preprint arXiv:2602.54321, 2026.

- [86] M. Zhu, et al., "Prompt Injection Attack to Tool Selection in LLM Agents," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2026.
- [87] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISeC), 2023. arXiv ID: 2302.12173.
- [88] K. Greshake, et al., "Not What You've Signed Up For," arXiv preprint arXiv:2302.12173, 2023.
- [89] H. Chang, et al., "Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems," arXiv preprint arXiv:2601.07072, 2026.
- [90] L. Wermke, et al., "WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks," in Proceedings of the International Conference on Learning Representations (ICLR), 2026.
- [91] S. Luong, et al., "Trust Me, I Know This Function: Hijacking LLM Static Analysis using Bias," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2026.
- [92] Y. Liu, G. Deng, Z. Xu, and Y. Li, "Securing AI Agents Against Prompt Injection Attacks," arXiv preprint arXiv:2503.04567, 2025.
- [93] Z. Cheng, et al., "Attention is All You Need to Defend Against Indirect Prompt Injection Attacks in LLMs," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2026.
- [94] A. Popovic, et al., "Prompt Injection Detection and Mitigation via AI Multi-Agent NLP Frameworks," arXiv preprint arXiv:2503.11517, 2025.
- [95] E. Debenedetti, et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," arXiv preprint arXiv:2406.13352, 2024.
- [96] J. Zhang, et al., "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents," arXiv preprint, 2024.
- [97] M. Zhu, et al., "HouYi: Black-box Prompt Injection Attacks on LLM-Integrated Applications," arXiv preprint, 2023.
- [98] M. Zhu, et al., "ToolHijacker: Prompt Injection Attack to Tool Selection in LLM Agents," NDSS, 2026.
- [99] M. Parikh, "Static Analysis for LLM Prompt Security: A Methodology for Pre-Deploy Vulnerability Detection," DEV Community, 2026.
- [100] Z. Cheng, et al., "AlignSentinel: Attention-based Detection against Prompt Injection," NDSS, 2026.
- [101] X. Yi, et al., "PI-Hunter: Automated Red-Teaming for Exposing and Localizing Prompt Injections," arXiv preprint arXiv:2606.12737, 2026.
- [102] H. Chen, et al., "SecureForge: Finding and Preventing Vulnerabilities in LLM-Generated Code via Prompt Optimization," arXiv preprint arXiv:2605.08382, 2026.
- [103] O. F. Khan Suri, et al., "Securing Large Language Models from Prompt Injection Attacks," arXiv preprint arXiv:2512.01326, 2025.
- [104] F. Perez, et al., "Ignore Previous Prompt," arXiv preprint arXiv:2211.09527, 2022.